1.er trimestre · C# · WPF · Named Pipes

# Desafío PSP y DIN

Guía del trabajo práctico de Programación de Servicios y Procesos y Diseño de Interfaces: qué hay que construir, qué hay que entregar y en qué orden conviene hacerlo.

Pareja: David y Diego | Ámbito: taller mecánico

## 00 Uso de la IA

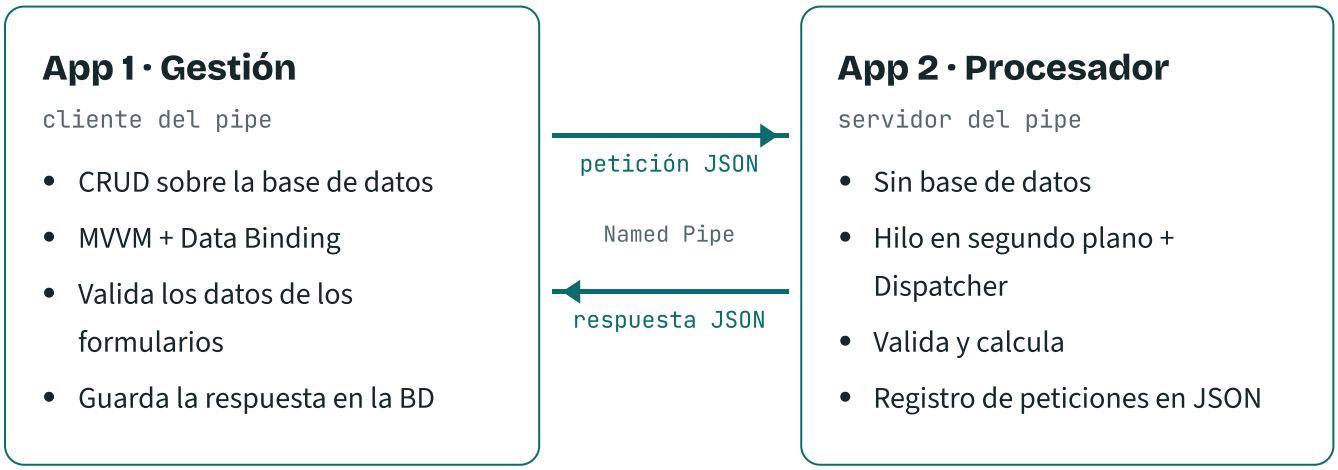
**REGLA DEL ENUNCIADO**

No se puede usar IA (ChatGPT, Copilot…) para generar el trabajo, ni entero ni en parte. Solo se permite como apoyo para entender conceptos o resolver dudas, siempre de forma responsable y transparente.

En la defensa pueden preguntar por **cualquier parte del código**, oralmente o por escrito. Si no sabemos explicarlo, el trabajo se califica como suspenso.

## 01 Visión general

En pareja, hay que desarrollar un sistema de gestión formado por **dos aplicaciones de escritorio independientes en C# con WPF**. La primera hace CRUD sobre una base de datos relacional y la segunda procesa los datos. Se comunican mediante **Named Pipes**, intercambiando mensajes en **JSON**.




Nuestro ámbito es un **taller mecánico**. El enunciado exige que **lo apruebe el profesor**, así que es lo primero que hay que confirmar.

El ejemplo del enunciado es un laboratorio que mide glucosa, colesterol y triglicéridos. En un taller, lo equivalente sería una **revisión de vehículo**: la App 1 envía las mediciones tomadas (presión de neumáticos, grosor de las pastillas de freno, profundidad del dibujo…) y la App 2 las clasifica. Los ejemplos de esta guía siguen esa idea, pero son solo una propuesta: el diseño final lo decidimos nosotros.

---

# 02 Entregables

| ENTREGABLE | QUÉ INCLUYE |
|---|---|
| **1 · Código** | Las dos aplicaciones, documentadas con comentarios escritos por nosotros. |
| **2 · SQL** | Scripts con `CREATE TABLE` e `INSERT INTO` para crear la BD y cargar los datos de inicio. |
| **3 · JSON** | Los archivos que se envían y se reciben entre las dos apps, y el JSON del registro de peticiones que lee y escribe la App 2. |
| **4 · Documento** | Memoria del proyecto con capturas y esquemas (ver apartado 06). |
| **GitHub** | Repositorio **público** con todos los cambios. El enlace se sube a Sallenet. |
| **Defensa** | La app se revisa ejecutándose en directo en nuestro ordenador, con preguntas sobre el código. |

---

# 03 App 1 · Sistema de gestión

Es la aplicación principal. Gestiona la información con una interfaz WPF, valida los datos que se introducen, envía peticiones de procesamiento a la App 2 y guarda los resultados en la base de datos para poder consultarlos después.

## Base de datos relacional

Posibles tablas para el taller:

`Clientes` `Vehiculos` `Mecanicos` `Revisiones` `Comprobaciones` `Revision_Comprobacion`

| REQUISITO | QUÉ SIGNIFICA |
| --- | --- |
| **4 tablas mínimo** | Todas relacionadas entre sí. |
| **Relación 1:N** | Al menos una. Ejemplo: un cliente tiene varios vehículos, y un vehículo tiene varias revisiones. |
| **Relación N:M** | Al menos una, con **tabla intermedia** que tenga un atributo propio además de las dos claves. Ejemplo: una revisión incluye varias comprobaciones y una comprobación aparece en muchas revisiones: `Revision_Comprobacion(id_revision, id_comprobacion, valor_medido)`. |
| **PK y FK** | Claves primarias y foráneas bien definidas. |
| **ON DELETE / ON UPDATE** | Usar `CASCADE` y/o `SET NULL` donde tenga sentido. Hay que saber justificar cada elección. Ejemplos: si se borra un vehículo, sus revisiones se borran en `CASCADE`; si se da de baja a un mecánico, sus revisiones se conservan con el mecánico a `SET NULL`. |
| **Resultados** | Lo que devuelve la App 2 se guarda en la BD. |
| **Datos de prueba** | Suficientes y coherentes para probar el CRUD y el envío de peticiones. |

## Arquitectura MVVM

- **Model**: las entidades, una clase por tabla.
- **ViewModel**: la lógica y los comandos (`ICommand`). Implementa `INotifyPropertyChanged`.
- **View**: el XAML. Nada de acceder a los controles desde el code-behind (`txtNombre.Text = …`): todo va por Data Binding.

## Operaciones obligatorias

1. **Crear**, validando antes de guardar. En el taller, por ejemplo: la matrícula y el DNI del cliente no pueden repetirse, el kilometraje no puede ser negativo, el año de fabricación no puede ser futuro y el email tiene que tener un formato válido.
2. **Modificar**, validando también y teniendo en cuenta el `ON UPDATE` de las tablas relacionadas.
3. **Cargar** los datos.
4. **Borrar**, teniendo en cuenta el `ON DELETE`.

5. **Gestionar la selección del DataGrid**: al elegir una fila, sus datos aparecen en el formulario.

## Interfaz

No basta con que funcione: también se evalúa cómo se presenta y cómo se usa. Hay que cuidar márgenes, alineaciones y tamaños, y que los textos sean legibles y estén bien etiquetados. El binding tiene que ser **TwoWay**, usar **SelectedItem** y actualizar la interfaz de forma automática.

| TIPO | CONTROLES OBLIGATORIOS |
|---|---|
| **Entrada** | `TextBox` `DatePicker` `ComboBox (FK)` `CheckBox` `Button` `ListBox` `PasswordBox` |
| **Visualización** | `DataGrid` |
| **Organización** | `Grid` `StackPanel` `DockPanel` `GroupBox` `Border` |
| **Comunes** | `Label` `Image` `Menu` |

Ideas para encajar los controles en el taller:

- `ComboBox`: elegir el cliente al dar de alta un vehículo, o el mecánico de una revisión.
- `DatePicker`: fecha de entrada del vehículo o de la revisión.
- `CheckBox`: «ITV en vigor» o «Revisión urgente».
- `ListBox`: las comprobaciones incluidas en una revisión.
- `PasswordBox`: acceso del mecánico al iniciar la aplicación.
- `Image`: el logotipo del taller.

**OJO CON EL PASSWORDBOX**

Su propiedad `Password` no admite binding directo. Hay que pensar cómo usarlo dentro de MVVM (por ejemplo, en una pantalla de login) y saber explicar la solución en la defensa.

### Comunicación (lado cliente)

Serializa la petición a JSON, la envía con `NamedPipeClientStream`, recibe la respuesta, la deserializa y guarda el resultado en la base de datos.

---

## 04 App 2 · Servicio de procesamiento

Es una aplicación WPF independiente **sin base de datos** que procesa las peticiones y sirve para supervisarlas. No es un CRUD.

### Parte 1 · Named Pipes (lado servidor)

1. Recibe los datos serializados con `NamedPipeServerStream`.
2. Los deserializa, los valida y los procesa, haciendo **cálculos internos**. En el taller, por ejemplo, comparar cada medición con su límite y clasificarla como «Correcto», «Revisar» o «Sustituir».
3. Registra la petición y publica los eventos que tiene que mostrar la interfaz: peticiones, estados y errores.
4. Devuelve la respuesta serializada a la App 1.

### Parte 2 · Hilos y Dispatcher (obligatorio)

El servidor del pipe tiene que ejecutarse en un **hilo en segundo plano** para que la ventana no se congele. Ese hilo no puede tocar la interfaz directamente, así que las actualizaciones pasan por el Dispatcher:

```
await Dispatcher.InvokeAsync(() ⇒
{
    dgPeticiones.Items.Add("Petición recibida");
});
```

Este es el fragmento literal del enunciado. Con MVVM lo normal es añadir el registro a la `ObservableCollection` enlazada al DataGrid, en lugar de usar `Items.Add`.

### Parte 3 · Persistencia en JSON

El registro de peticiones se guarda en un archivo JSON y **se carga al abrir la aplicación**.

### Pantalla: Registro de peticiones

Historial de las solicitudes recibidas, en un `DataGrid` enlazado a `ObservableCollection<PeticionRegistro>`. Al seleccionar una fila se abre el **detalle** de esa petición.

| COLUMNA | CONTENIDO |
| --- | --- |
| **ID de petición** | El que envía la App 1, por ejemplo `REV-2026-001`. |
| **Fecha y hora** | Momento de la recepción. |
| **Tipo de operación** | Por ejemplo neumáticos, frenos o aceite. |
| **Estado final** | Correcto, o error de validación si algún dato no es lógico. |
| **Error técnico** | El mensaje de la excepción, si la hubo. |

**ESTADO Y ERROR SON COSAS DISTINTAS**

Una petición puede terminar con el estado «Error de validación» y mostrar en el detalle el campo concreto que falla, por ejemplo «El kilometraje no puede ser negativo» o «La presión de NEU no puede ser 0 bar». El error técnico se reserva para las excepciones.

### Pantalla: Acerca de

Nombre de la aplicación, integrantes de la pareja (David y Diego), fecha de creación y versión del programa.

---

## 05 Formato JSON

El enunciado trae un ejemplo de laboratorio. Esta es la misma estructura adaptada a una revisión del taller, como punto de partida para decidir la nuestra. Conviene fijar el formato **antes** de programar, porque es el contrato entre las dos aplicaciones.

App 1 → App 2 · petición

```
{
  "idSolicitud": "REV-2026-001",
  "vehiculo": {
    "matricula": "1234 KLM",
    "kilometraje": 98500,
    "itvEnVigor": true
  },
  "comprobaciones": [
    { "codigo": "NEU", "valorMedido":
2.2,
      "unidad": "bar" },
    { "codigo": "FRE", "valorMedido":
2.5,
      "unidad": "mm" },
    { "codigo": "DIB", "valorMedido":
1.4,
      "unidad": "mm" }
  ]
}
```

App 2 → App 1 · respuesta

```
{
  "idSolicitud": "REV-2026-001",
  "estado": "Procesada correctamente",
  "resultados": [
    { "codigo": "NEU", "valorMedido":
2.2,
      "unidad": "bar",
      "clasificacion": "Correcto" },
    { "codigo": "FRE", "valorMedido":
2.5,
      "unidad": "mm",
      "clasificacion": "Revisar" },
    { "codigo": "DIB", "valorMedido":
1.4,
      "unidad": "mm",
      "clasificacion": "Sustituir" }
  ]
}
```

| CÓDIGO | COMPROBACIÓN | UNIDAD |
|---|---|---|
| **NEU** | Presión de los neumáticos | bar |
| **FRE** | Grosor de las pastillas de freno | mm |
| **DIB** | Profundidad del dibujo del neumático (el mínimo legal en España es 1,6 mm) | mm |

Los umbrales de cada clasificación los tenemos que definir nosotros y saber explicarlos.

**ERRATA EN EL PDF**

En el JSON de respuesta del enunciado falta el `]` que cierra `resultados`. Si se copia el original tal cual, no se podrá deserializar.

---

## 06 Documento del proyecto

Estructura que pide el enunciado. Lo ideal es ir rellenándola mientras se trabaja, no al final.

### 1. Descripción del proyecto

En qué consiste, cuál es su objetivo y qué se pretende conseguir.

### 2. Partes del proyecto

Los componentes principales: App 1, App 2, base de datos, comunicación por Named Pipes y archivos JSON.

### 3. Proceso de desarrollo

Las tareas paso a paso y en orden cronológico, indicando quién hizo cada parte. El historial de commits sirve de apoyo.

### 4. Imágenes

Capturas de las pantallas, diagrama entidad-relación, esquema de la comunicación… Cada imagen con una breve explicación.

### 5. Dificultades y soluciones

Los problemas que aparecieron y cómo se resolvieron.

**CONSEJO**

Apuntad las dificultades en el momento en que aparecen (aunque sea en una nota rápida) y haced capturas de cada fase. Reconstruir todo eso al final cuesta mucho más.

## 07 Plan de trabajo

Orden sugerido. El reparto es orientativo y se puede intercambiar: aunque cada uno lleve una app, **los dos tenemos que entender las dos**, porque en la defensa pueden preguntar cualquier parte.

| FASE | TAREA | QUIÉN |
|---|---|---|
| 1 | Elegir el ámbito y pedir la aprobación al profesor. | `Los dos` |
| 2 | Crear el repositorio público en GitHub y la solución con los dos proyectos WPF. | `Los dos` |
| 3 | Diseñar la BD (diagrama E-R), escribir los scripts SQL y los datos de prueba. | `Diego` |
| 4 | Acordar el formato JSON de petición y respuesta. | `Los dos` |
| 5 | App 1: models, viewmodels, CRUD y validaciones. | `Diego` |
| 6 | App 2: servidor del pipe en un hilo, validación, cálculos y Dispatcher. | `David` |
| 7 | App 2: guardar y cargar el registro JSON, ventana de detalle y «Acerca de». | `David` |
| 8 | App 1: cliente del pipe y guardado de la respuesta en la BD. | `Los dos` |
| 9 | Pulir la interfaz y revisar que están todos los controles obligatorios. | `Los dos` |
| 10 | Comentar el código, terminar el documento y ensayar la defensa. | `Los dos` |

## 08 Checklist final

Para repasar antes de entregar.

- [ ] Ámbito (taller mecánico) aprobado por el profesor
- [ ] BD con 4 tablas o más, una relación 1:N y una N:M
- [ ] Tabla intermedia N:M con un atributo propio
- [ ] ON DELETE y ON UPDATE definidos y justificados
- [ ] Scripts SQL con CREATE TABLE e INSERT INTO
- [ ] App 1 en MVVM, con INotifyPropertyChanged y comandos
- [ ] Crear, modificar, cargar y borrar funcionando, con validaciones
- [ ] Selección del DataGrid con SelectedItem y binding TwoWay
- [ ] Todos los controles obligatorios presentes
- [ ] App 1 envía y recibe JSON por Named Pipe y guarda el resultado en la BD
- [ ] App 2 recibe, valida, calcula y responde
- [ ] App 2 usa un hilo en segundo plano y el Dispatcher
- [ ] Registro de peticiones con sus 5 columnas y ventana de detalle
- [ ] Registro guardado en JSON y cargado al iniciar la App 2
- [ ] Pantalla «Acerca de» con nombre, autores, fecha y versión
- [ ] Archivos JSON de ejemplo incluidos en la entrega
- [ ] Código comentado por nosotros
- [ ] Documento con sus 5 apartados y las imágenes explicadas
- [ ] Repositorio público en GitHub y enlace subido a Sallenet
- [ ] Los dos sabemos explicar cualquier parte del código